

SaveSmart: Adaptive Budgeting Agent

Problem framing and persona:

Persona Profile: Rohan is a junior software developer working at a startup in Pune. He earns Rs 45,000 per months in hand that is credited on 1st of every month and is trying to save Rs 1,50,000 in 6 months for his first bike Royal Enfield Classic 350. Currently, he has saved Rs 47,000 over the last 3 months and is aiming to save Rs 25000 each month for the next 6 months.

Behavior: Eats out frequently. Does occasional shopping.

Pain points: He has planned expenses per month worth Rs 26000 but aims to save Rs 25000 per month while his salary is Rs 45000. This is mathematically impossible without behavior change.

Agentic AI reasoning: Traditional App tracks the spending but SaveSmart replans decisions in real time when constraints change.

Persona Profile

Name & Role	Rohan - Junior Software Developer, Pune startup
Income	₹45,000/month in-hand, credited on 1st of every month
Goal	Save ₹1,50,000 in 6 months for Royal Enfield Classic 350 (₹25,000/month target)
Current Status	Month 3 of 6 - ₹47,000 saved so far across 3 months
Behaviour	Eats out frequently (Swiggy, Zomato). Does occasional shopping. Peer-pressure spender.

Core Pain Point	Planned monthly expenses ₹26,000 + savings goal ₹25,000 = ₹51,000. Income ₹45,000. Mathematically impossible without real-time behaviour change.
------------------------	--

Problem Statement

Traditional apps track spending - SaveSmart replans decisions in real time when constraints change. Users don't need awareness. They need replacement decisions at the moment of disruption.

Agentic AI Reasoning - Why Not a Traditional App?

Traditional Budgeting App	SaveSmart - Agentic AI
Tracks what you spent - rearview mirror	Replans what to do next
No response when emergency hits	Emergency triggers live multi-step agent reasoning
Generic advice: 'spend less on food'	Specific: 'Skip Swiggy for 14 days, save ₹1,900'
User figures out the recovery plan alone	3 ranked trade-off scenarios - user just picks one

UX flow and interface design:

User journey:



Here is the figma link for UX flow: <https://bust-distort-37683080.figma.site/>

Backend logic: Here is the N8N link:

<https://victor2218.app.n8n.cloud/workflow/faka9b5qOvdiJiUJ?projectId=fk4g8fZGb1mutR8E>

Agent Logic Note: So N8N workflow first takes data from Lovable and then calculates gap between expected savings and actual savings using calculation node. Then Scoring node evaluates the score of each and every spend to evaluate which one should be prioritized or reduced. Finally scenario node decides the final recommendation in 3 scenarios and recommends the best one.

JSON file: {

```
"name": "SaveSmart_Hackathon",  
"nodes": [  
  {  
    "parameters": {  
      "httpMethod": "POST",  
      "path": "agent-run",  
      "responseMode": "lastNode",  
      "options": {}  
    },  
    "type": "n8n-nodes-base.webhook",  
    "typeVersion": 2.1,  
    "position": [  
      0,  
      0  
    ],  
    "id": "58b76bbe-9520-415e-b964-bdabb5baf51e",  
    "name": "Webhook",
```

```

    "webhookId": "9772827f-e531-4955-b58b-93606a543856"

  },

  {

    "parameters": {

      "jsCode": "const raw = $input.first().json;\n\n// Extract body safely\nconst input = raw.body\n|| raw;\n\n// Safety validation\nif (!input.transactions || !input.income ||\n!input.monthly_savings_target) {\n  return [\n    {\n      json: {\n        error: \"Missing required\nfields\", \n        received: input\n      }\n    }];\n}\n\n// Calculate total spent\nlet totalSpent =\n0;\nfor (const tx of input.transactions) {\n  if (tx.amount < 0) {\n    totalSpent +=\nMath.abs(tx.amount);\n  }\n}\n\n// Remaining cash\nconst remainingCash = input.income -\ntotalSpent;\n\n// Gap\nconst gap = input.monthly_savings_target - remainingCash;\n\n// Return\nenriched data\nreturn [\n  {\n    json: {\n      ...input,\n      total_spent: totalSpent,\n      remaining_cash: remainingCash,\n      gap: gap\n    }\n  }];\n"

    },

    "type": "n8n-nodes-base.code",

    "typeVersion": 2,

    "position": [

      208,

      0

    ],

    "id": "1c9a6ecf-7b5b-4a02-a9e1-210281065ce6",

    "name": "Calculation node"

  },

  {

    "parameters": {

      "jsCode": "const data = $input.first().json;\n\n// Safety check\nif (!data.transactions ||\n!data.budget) {\n  return [\n    {\n      json: {\n        error: \"Missing transactions or budget\", \n        received: data\n      }\n    }];\n}\n\n// Aggregate category data\nconst categoryMap = {};\nfor\n(const tx of data.transactions) {\n  if (tx.amount < 0) {\n    const cat = tx.category;\n    if\n(!categoryMap[cat]) {\n      categoryMap[cat] = { frequency: 0, amount: 0 };\n    }\n  }\n}

```

```

categoryMap[cat].frequency += 1;\n  categoryMap[cat].amount += Math.abs(tx.amount);\n
}\n}\n\n// Flexibility weights\nconst weights = {\n  Need: 0.1,\n  Want: 1.0,\n  Buffer: 0.6\n};\n\n// Map types from budget\nfor (const b of data.budget) {\n  if (categoryMap[b.category]) {\n    categoryMap[b.category].type = b.type;\n  }\n}\n\n// Compute scores\nconst scores = [];\nfor (const cat in categoryMap) {\n  const freq = categoryMap[cat].frequency;\n  const amt = categoryMap[cat].amount;\n  const type = categoryMap[cat].type || \"Need\";\n  const weight = weights[type] || 0.1;\n  const score = Math.log(freq + 1) * amt * weight;\n  scores.push({\n    category: cat,\n    score: Math.round(score)\n  });\n}\n\n// Sort descending\nscores.sort((a, b) => b.score - a.score);\n\n// Return\nreturn [\n  {\n    json: {\n      ...data,\n      category_scores: scores\n    }\n  }\n];"

```

```

},

```

```

"type": "n8n-nodes-base.code",

```

```

"typeVersion": 2,

```

```

"position": [

```

```

  416,

```

```

  0

```

```

],

```

```

"id": "4c688e3e-26d1-4b9e-8210-160b6584c830",

```

```

"name": "Scoring node"

```

```

},

```

```

{

```

```

  "parameters": {

```

```

    "jsCode": "const data = $input.first().json;\n\n// Safety check\nif (typeof data.gap !==\n  \"number\") {\n  return [\n    {\n      json: {\n        error: \"Gap not computed\",\n        received: data\n      }\n    }\n  ];\n}\n\nconst gap = data.gap;\n\n// Scenarios (deterministic)\nconst scenarios = [\n  {\n    name: \"A — Monk Mode\",\n    recovery: 1920,\n    actions: [\n      \"Skip dining out (₹600)\",\n      \"Use misc buffer (₹1,320)\"\n    ],\n    impact: \"Low disruption\"\n  },\n  {\n    name: \"B — Balanced Cut\",\n    recovery: 4620,\n    actions: [\n      \"Limit dining out\",\n      \"Use misc buffer\",\n      \"Reduce next month wants (₹3,000)\"\n    ],\n    impact: \"Moderate disruption\"\n  },\n  {\n    name: \"C — Stretch Goal\",\n    recovery: 7000,\n    actions: [\n      \"Cut all wants\",\n      \"Use misc buffer\",\n      \"Freelance income push\"\n    ],\n    impact: \"High effort\"\n  }\n];\n\n// Recommendation logic\nlet recommended = scenarios[0];\n\nif (scenarios[2].recovery >= gap) {\n  recommended = scenarios[2];\n} else if

```

```
(scenarios[1].recovery >= gap * 0.6) {\n recommended = scenarios[1];\n}\n\n// Goal
status\nconst goal_status =\n recommended.recovery >= gap ? \"recovered\" : \"at_risk\";\n\n//
Final response\nreturn [\n {\n json: {\n total_spent: data.total_spent,\n remaining_cash:
data.remaining_cash,\n gap: data.gap,\n category_scores: data.category_scores,\n
scenarios: scenarios,\n recommended: recommended.name,\n goal_status: goal_status\n
}\n }\n];"
```

```
},
```

```
"type": "n8n-nodes-base.code",
```

```
"typeVersion": 2,
```

```
"position": [
```

```
624,
```

```
0
```

```
],
```

```
"id": "a64b3075-0cb9-4249-bcdc-e9e2f5f1e628",
```

```
"name": "Scenario"
```

```
}
```

```
],
```

```
"pinData": {},
```

```
"connections": {
```

```
"Webhook": {
```

```
"main": [
```

```
[
```

```
{
```

```
"node": "Calculation node",
```

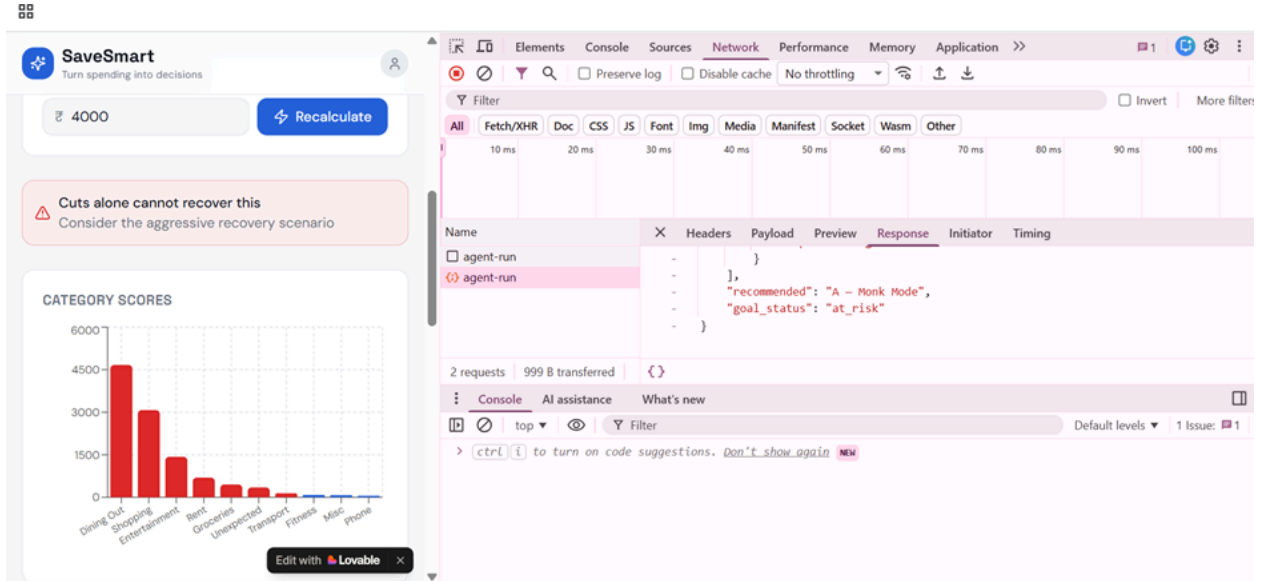
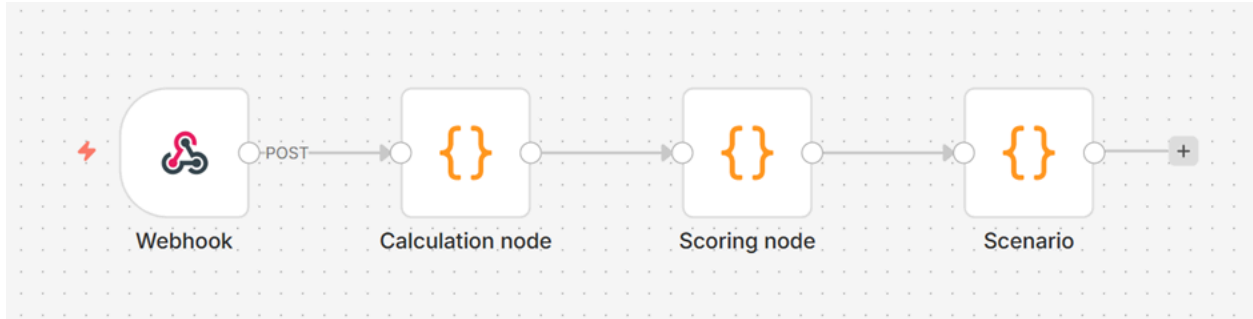
```
"type": "main",
```

```
"index": 0
```

```
}
```

```
    ]  
  ]  
},  
"Calculation node": {  
  "main": [  
    [  
      {  
        "node": "Scoring node",  
        "type": "main",  
        "index": 0  
      }  
    ]  
  ]  
},  
"Scoring node": {  
  "main": [  
    [  
      {  
        "node": "Scenario",  
        "type": "main",  
        "index": 0  
      }  
    ]  
  ]  
}
```

```
,
"Scenario": {
  "main": [
    []
  ]
},
"active": true,
"settings": {
  "executionOrder": "v1",
  "binaryMode": "separate"
},
"versionId": "bf91f5eb-1b96-4922-8066-297146f6b1ca",
"meta": {
  "templateCredsSetupCompleted": true,
  "instanceId": "b354da00e0bcd914d14409fa9557dc4a7d9faecc7ac9f916f98fc192f997f59f"
},
"id": "faka9b5qOvdiJiUJ",
"tags": []
}
```



Live Prototype link: <https://save-track-plan.lovable.app/>

Success metric and decision log:

KPIs:

- Savings adherence rate
- Overshoot events prevented
- Trade-off acceptance rate
- Stress reduction (decision time ↓)

Success KPIs

KPI	What It Measures
-----	------------------

Savings Adherence Rate	% of months where Rohan hit his ₹25,000 savings target
Overshoot Events Prevented	Count of emergency shocks where agent plan was accepted and followed
Trade-off Acceptance Rate	% of generated scenarios that user accepted (target ≥ 60%)
Stress Reduction - Decision Time	Time from emergency input to plan accepted (target < 30 seconds)

Prompt Iteration Log

V	Change Made	Why & Impact
V1	Generic: 'Give 3 savings tips for Rohan'	Output was vague - 'eat less outside'. No mention of actual budget remaining. Unusable.
V2	Added full budget_state JSON to prompt	Claude used real numbers but returned markdown prose. Parse node failed 60% of runs.
V3	Strict JSON schema + 'Return ONLY valid JSON' in system prompt	100% parse success. Scenarios now specific (₹600 dining left, ₹1,320 buffer available). Fallback node never triggered.

Users don't need awareness.
They need **replacement decisions at the moment of disruption**.

Pitch Deck:

<https://notebooklm.google.com/notebook/12bcb19d-595b-4ef1-b6bc-e6e814fc6150/artifact/0e0d1b21-bc40-483f-81ef-f6c82268749c>